example_1

April 26, 2026

1 Example 1

[3]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

[4]:
```
np.random.seed(42)
n_months = 200
dates = pd.date_range('2008-01-01', periods=n_months, freq='MS')
trend = np.linspace(50, 70, n_months)
seasonal = 10 * np.sin(2 * np.pi * np.arange(n_months) / 12)
noise = np.random.randn(n_months) * 2
y = trend + seasonal + noise
df = pd.DataFrame({'value': y}, index=dates)
```

[5]:
```
df.head()
```

[5]:

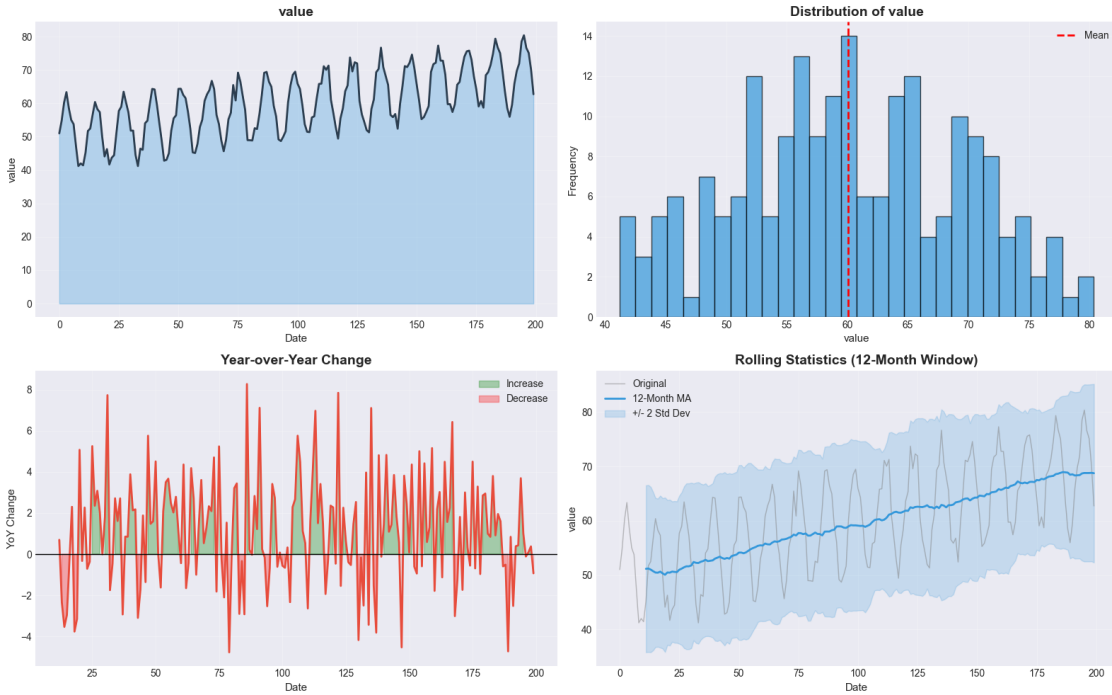
	value
2008-01-01	50.993428
2008-02-01	54.823974
2008-03-01	60.156636
2008-04-01	63.347567
2008-05-01	58.593957

1.1 EDA

[6]:
```
from fcstmetrics.utils import plot_eda
```

[7]:
```
fig = plot_eda(df['value'])
plt.show()
```

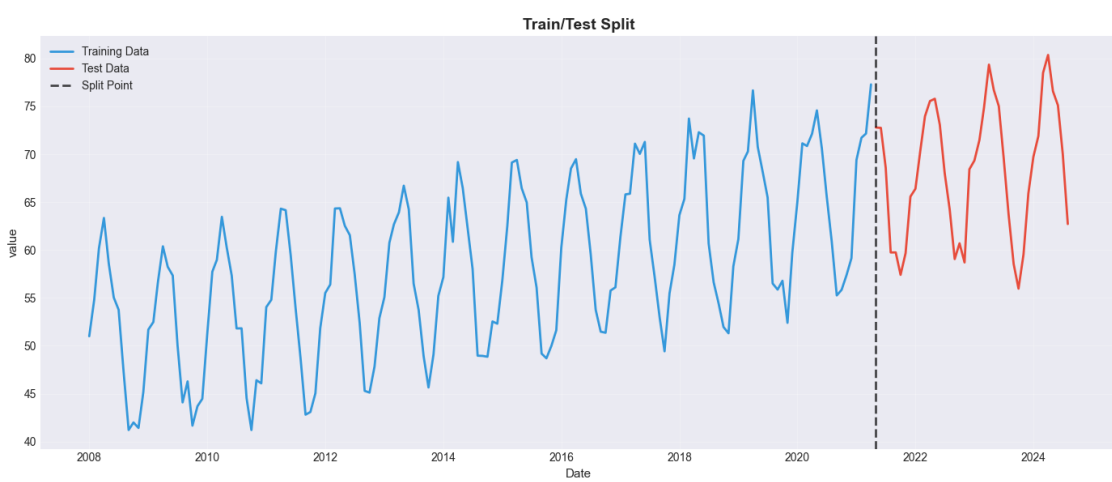
```
Mean value: 60.08%
Min value: 41.20% (2010-10-01 00:00:00)
Max value: 80.37% (2024-04-01 00:00:00)
Standard deviation: 9.26%
Inferred freq: <MonthBegin>
```



1.2 sktime formatting

[8]:
```
from fcstmetrics.utils import prepare_sktime_train_test
```

[9]:
```
y_train, y_test = prepare_sktime_train_test(data=df['value'], train_size=0.8,␣
↪freq='MS', start_date=df.index[0], include_plot = True)
```



[10]:
```
y_train = y_train.asfreq('MS')
y_test = y_test.asfreq('MS')
```

[11]:
```
print(f"Training: {len(y_train)} months")
print(f"Test: {len(y_test)} months")
print(f"Train period: {y_train.index[0]} to {y_train.index[-1]}")
print(f"Test period: {y_test.index[0]} to {y_test.index[-1]}")
```

```
Training: 160 months
Test: 40 months
Train period: 2008-01-01 00:00:00 to 2021-04-01 00:00:00
Test period: 2021-05-01 00:00:00 to 2024-08-01 00:00:00
```

1.3 Train models

[12]:
```
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from sklearn.linear_model import LinearRegression
```

[13]:
```
model_es = ExponentialSmoothing(y_train, seasonal_periods=12, trend='add',␣
↪seasonal='add', damped_trend=True)
fit_es = model_es.fit()
pred_es_train = fit_es.fittedvalues
pred_es_test = fit_es.forecast(len(y_test))
pred_es_test = pd.Series(pred_es_test, index=y_test.index)
```

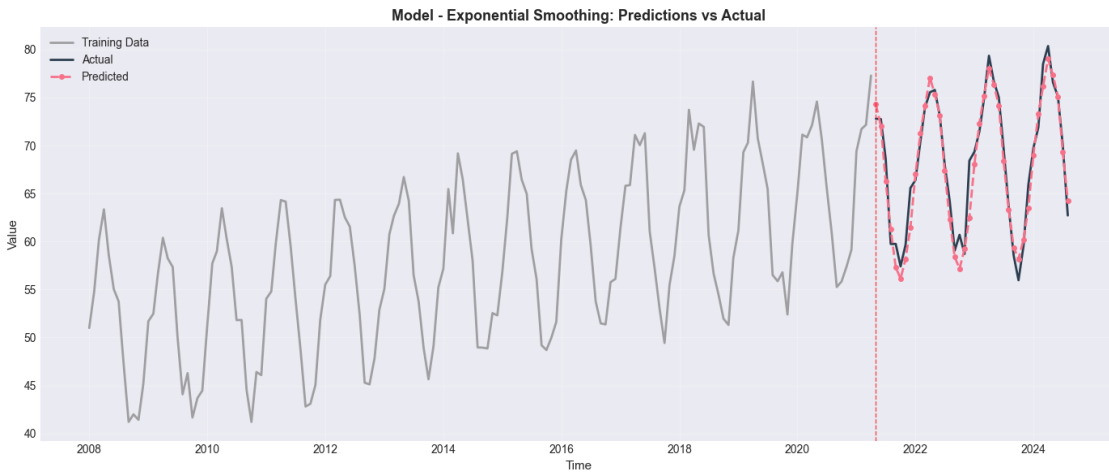
[14]:
```
X_train = np.column_stack([np.arange(len(y_train)),
                           np.sin(2 * np.pi * np.arange(len(y_train)) / 12),
                           np.cos(2 * np.pi * np.arange(len(y_train)) / 12)])
X_test = np.column_stack([np.arange(len(y_train), len(y_train) + len(y_test)),
                          np.sin(2 * np.pi * np.arange(len(y_train),␣
↪len(y_train) + len(y_test)) / 12),
                          np.cos(2 * np.pi * np.arange(len(y_train),␣
↪len(y_train) + len(y_test)) / 12)])
lr = LinearRegression()
lr.fit(X_train, y_train.values)
pred_lr_train = lr.predict(X_train)
pred_lr_test = lr.predict(X_test)
pred_lr_test = pd.Series(pred_lr_test, index=y_test.index)
```

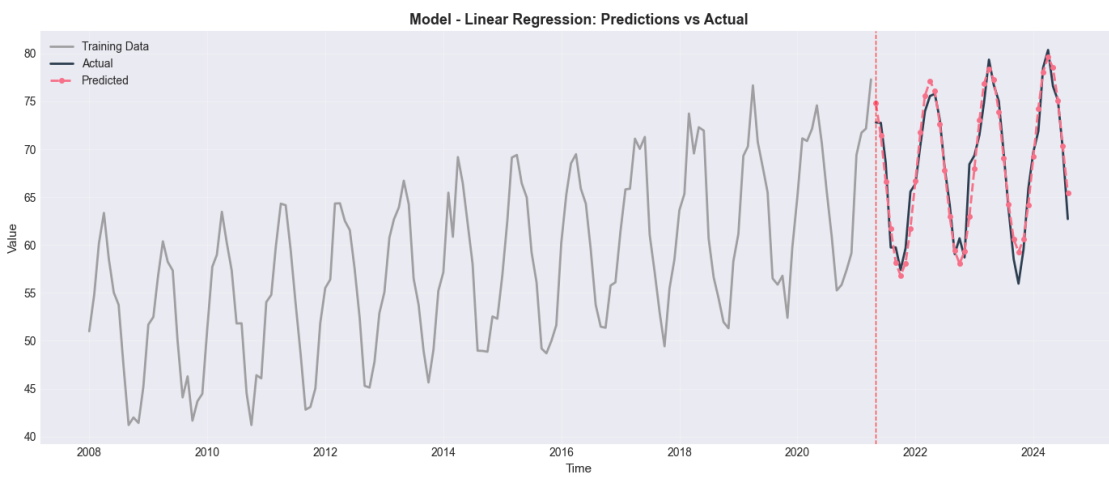
1.4 Plot predictions

[15]:
```
from fcstmetrics.utils import plot_predictions, plot_predictions_full
```

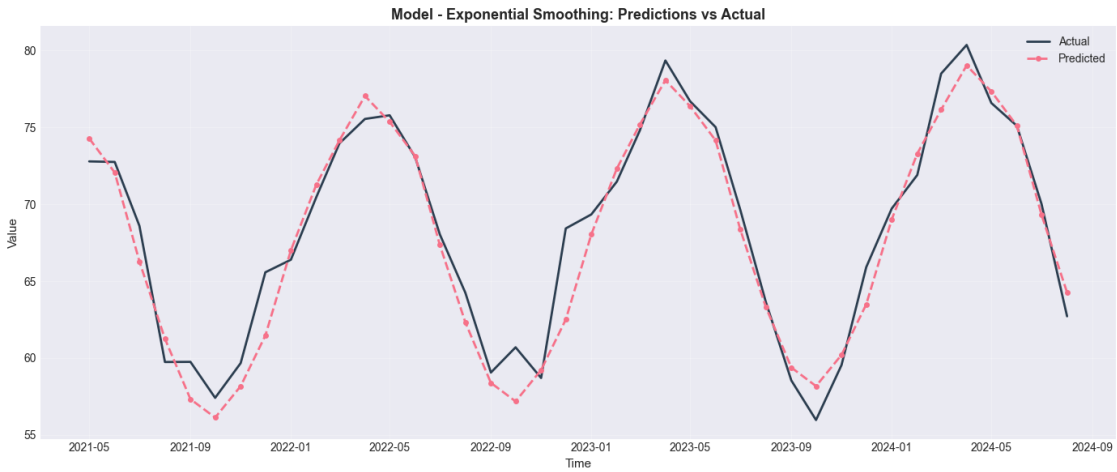
[16]:
```
models = {'Exponential Smoothing': pred_es_test,
          'Linear Regression': pred_lr_test}
for name, predictions in models.items():
```

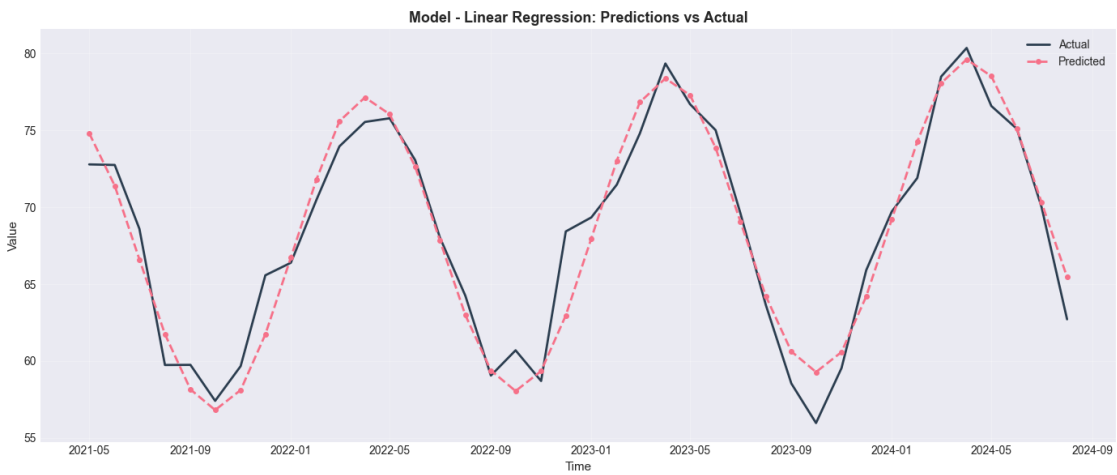
```
    fig = plot_predictions_full(y_train=y_train, y_test=y_test,␣
↪y_pred=predictions, title=f'{name}: Predictions vs Actual')
    plt.show()
```





[19]:
```
for name, predictions in models.items():
    fig = plot_predictions(y_test=y_test, y_pred=predictions, title=f'{name}:␣
↪Predictions vs Actual')
    plt.show()
```

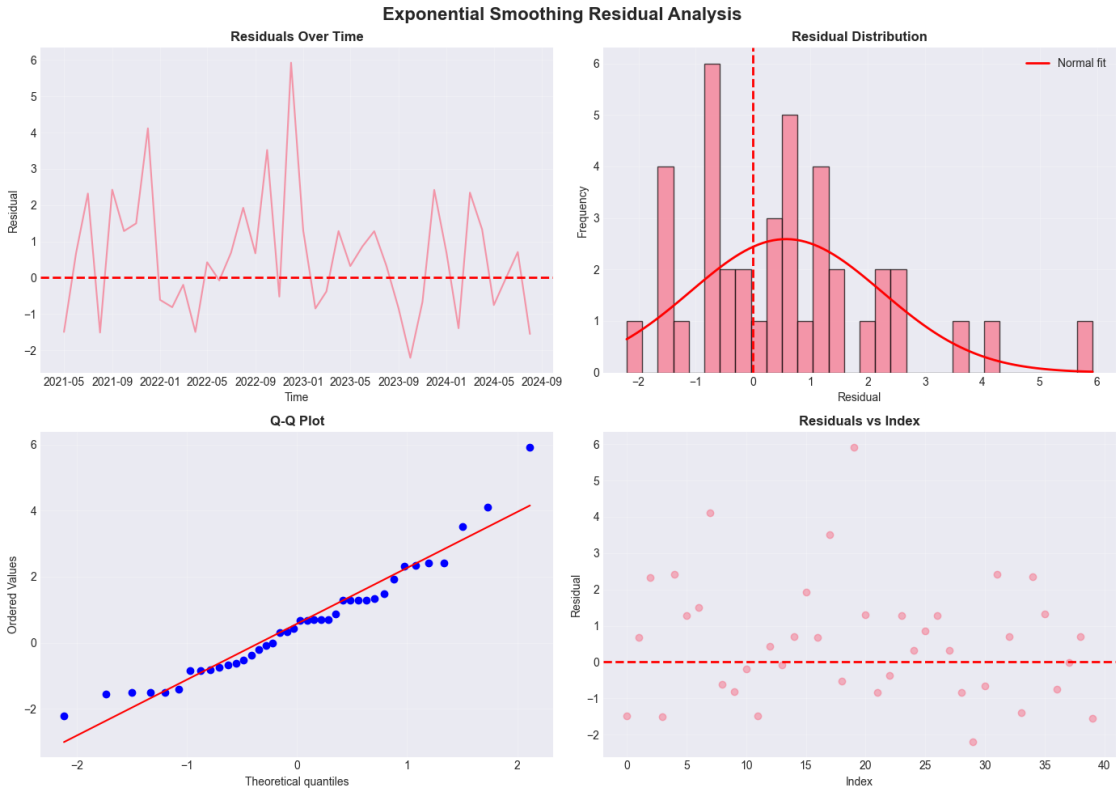




1.5 Residual analysis

[20]:
```
from fcstmetrics.utils import plot_residuals
```

[21]:
```
residuals_es = y_test.values - pred_es_test.values
fig = plot_residuals(residuals=residuals_es, timestamps=y_test.index,␣
↪title='Exponential Smoothing Residual Analysis')
plt.show()
```



1.6 Model Validation

[26]:
```
from fcstmetrics import validate_forecast, compare_models
import warnings
warnings.filterwarnings('ignore')
```

[27]:
```
report_es = validate_forecast(y_train=y_train.values,␣
↪y_train_pred=pred_es_train.values, y_test=y_test.values,
                              y_test_pred=pred_es_test.values,␣
↪model_name="Exponential Smoothing", print_report=True)
```

```
================================================================================
Evaluation Summary: Exponential Smoothing
================================================================================
Timestamp: 2026-04-23 22:12:56.610516

Metrics:
--------------------------------------------------------------------------------
  test_Bias                      :       0.5756
  test_Directional_Accuracy      :      89.7436
  test_MAE                       :       1.3431
  test_MAPE                      :       2.0199
```

```
  test_MASE                     :       0.3734
  test_MSE                      :       3.1199
  test_Max_Error                :       5.9242
  test_Mean_Error               :       0.5756
  test_Min_Error                :      -2.2057
  test_NRMSE                    :       0.2584
  test_R2                       :       0.9333
  test_RMSE                     :       1.7663
  test_Std_Error                :       1.6699
  train_Bias                    :       0.2219
  train_Directional_Accuracy    :      88.6792
  train_MAE                     :       1.4745
  train_MAPE                    :       2.5699
  train_MASE                    :       0.3933
  train_MSE                     :       3.3918
  train_Max_Error               :       4.2695
  train_Mean_Error              :       0.2219
  train_Min_Error               :      -3.9527
  train_NRMSE                   :       0.2151
  train_R2                      :       0.9538
  train_RMSE                    :       1.8417
  train_Std_Error               :       1.8283

Tests:
--------------------------------------------------------------------------------
  Pass | Jarque-Bera Normality: stat=1.4347, p=0.4880
  Pass | Shapiro-Wilk Normality: stat=0.9901, p=0.3251
  Pass | Kolmogorov-Smirnov Normality: stat=0.0376, p=0.9708
  Info | Ljung-Box Autocorrelation: stat=nan, p=N/A
  Pass | Durbin-Watson Autocorrelation: stat=2.1477, p=N/A
  Pass | Zero Mean Test: stat=1.5301, p=0.1280
  Pass | Homoscedasticity (Levene): stat=0.0354, p=0.8510
  Fail | Augmented Dickey-Fuller: stat=0.2703, p=0.9759
  Fail | KPSS Stationarity: stat=1.4769, p=0.0100

================================================================================
```

[28]:
```
report_lr = validate_forecast(y_train=y_train.values,␣
↪y_train_pred=pred_lr_train, y_test=y_test.values,
                              y_test_pred=pred_lr_test.values,␣
↪model_name="Linear Regression", print_report=True)
```

```
================================================================================
Evaluation Summary: Linear Regression
================================================================================
Timestamp: 2026-04-23 22:13:40.498517

Metrics:
```

```
--------------------------------------------------------------------------------
  test_Bias                      :      -0.0094
  test_Directional_Accuracy      :      89.7436
  test_MAE                       :       1.4275
  test_MAPE                      :       2.1442
  test_MASE                      :       0.3969
  test_MSE                       :       3.2347
  test_Max_Error                 :       5.4605
  test_Mean_Error                :      -0.0094
  test_Min_Error                 :      -3.3174
  test_NRMSE                     :       0.2631
  test_R2                        :       0.9308
  test_RMSE                      :       1.7985
  test_Std_Error                 :       1.7985
  train_Bias                     :      -0.0000
  train_Directional_Accuracy     :      88.6792
  train_MAE                      :       1.4880
  train_MAPE                     :       2.6038
  train_MASE                     :       0.3968
  train_MSE                      :       3.4325
  train_Max_Error                :       4.8533
  train_Mean_Error               :      -0.0000
  train_Min_Error                :      -4.8533
  train_NRMSE                    :       0.2163
  train_R2                       :       0.9532
  train_RMSE                     :       1.8527
  train_Std_Error                :       1.8527

Tests:
--------------------------------------------------------------------------------
  Pass | Jarque-Bera Normality: stat=0.6217, p=0.7328
  Pass | Shapiro-Wilk Normality: stat=0.9952, p=0.8876
  Pass | Kolmogorov-Smirnov Normality: stat=0.0273, p=0.9996
  Info | Ljung-Box Autocorrelation: stat=nan, p=N/A
  Pass | Durbin-Watson Autocorrelation: stat=2.2005, p=N/A
  Pass | Zero Mean Test: stat=-0.0000, p=1.0000
  Pass | Homoscedasticity (Levene): stat=0.1697, p=0.6810
  Fail | Augmented Dickey-Fuller: stat=0.2703, p=0.9759
  Fail | KPSS Stationarity: stat=1.4769, p=0.0100

================================================================================
```

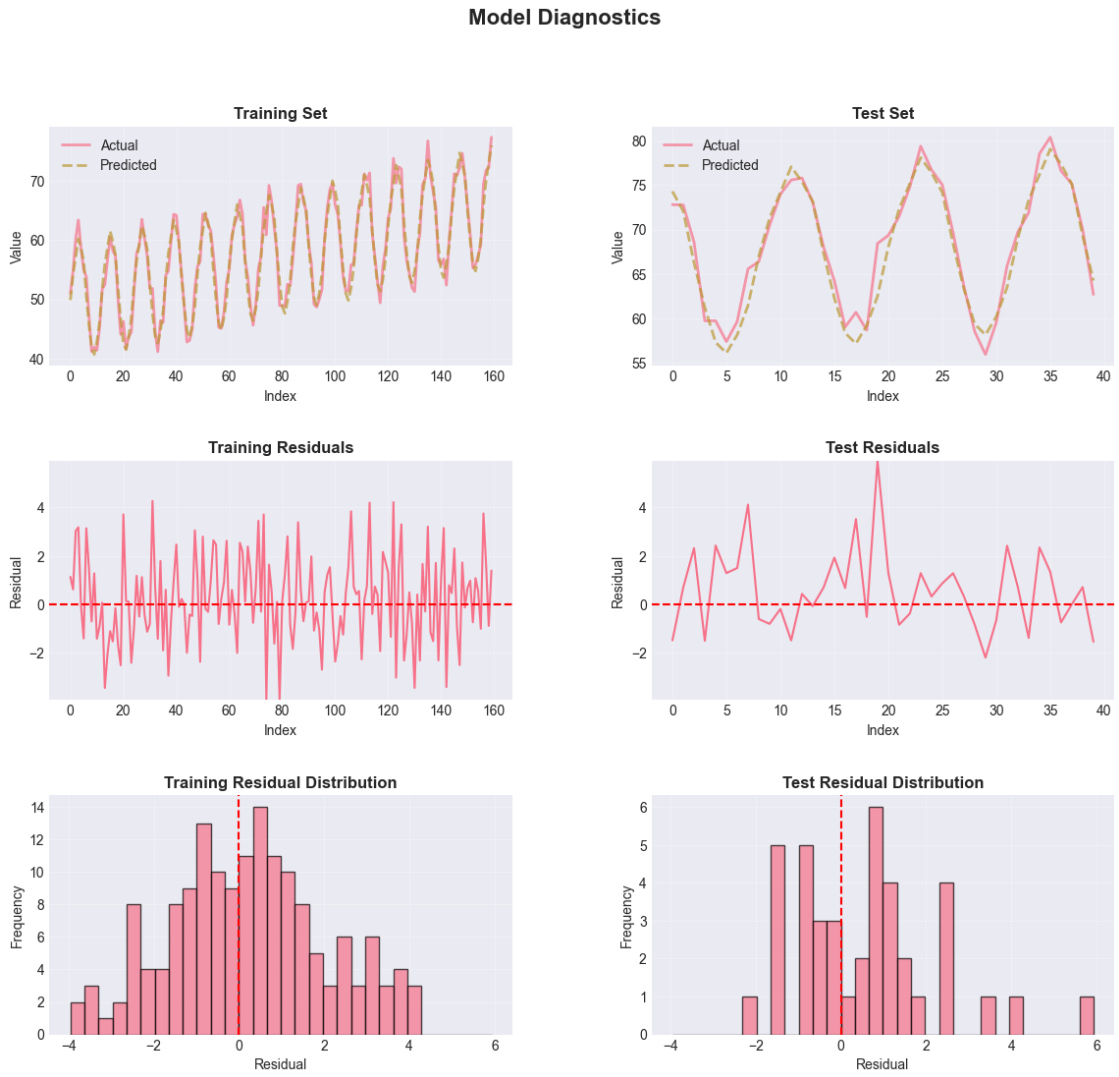
[29]:
```
models_comparison = {'ExpSmoothing': (y_train.values, pred_es_train.values,␣
 ↪y_test.values, pred_es_test.values),
                     'LinearReg': (y_train.values, pred_lr_train, y_test.
 ↪values, pred_lr_test.values)}
comparator = compare_models(models_comparison, print_report=True)
```

```
================================================================================
Model Comparison Summary
================================================================================

Metrics Comparison:
--------------------------------------------------------------------------------
               test_RMSE  test_MAE  test_MAPE  test_R2  test_Directional_Accuracy
test_RMSE_rank  test_MAE_rank  test_MAPE_rank  test_R2_rank
test_Directional_Accuracy_rank  avg_rank
ExpSmoothing     1.7663    1.3431     2.0199   0.9333                    89.7436
1.0            1.0             1.0           1.0                            1.5
1.1
LinearReg        1.7985    1.4275     2.1442   0.9308                    89.7436
2.0            2.0             2.0           2.0                            1.5
1.9

Best Model by Metric:
--------------------------------------------------------------------------------
  test_RMSE: ExpSmoothing (1.7663)
  test_MAE: ExpSmoothing (1.3431)
  test_MAPE: ExpSmoothing (2.0199)
  test_R2: ExpSmoothing (0.9333)
  test_Directional_Accuracy: ExpSmoothing (89.7436)

Overall Best Model (by avg rank): ExpSmoothing

================================================================================
Statistical Comparison Tests (Diebold-Mariano):
================================================================================

ExpSmoothing vs LinearReg:
  DM Statistic: -0.2820
  P-value: 0.7795
  Result: No significant difference
```

1.7 Diagnostic plots

[33]:
```
from fcstmetrics.utils import plot_diagnostics
```

[34]:
```
fig = plot_diagnostics(y_train=y_train.values, y_train_pred=pred_es_train.
↪values, y_test=y_test.values, y_test_pred=pred_es_test.values)
plt.show()
```


Model Diagnostics
Training Set
Actual
Predicted
Value
Index
Test Set
Actual
Predicted
Value
Index
Training Residuals
Residual
Index
Test Residuals
Residual
Index
Training Residual Distribution
Frequency
Residual
Test Residual Distribution
Frequency
Residual